Salesforce Project Documentation

WhatNext Vision Motors:

Shaping the Future of Mobility with Innovation and Excellence

Field	Detail
Prepared By	Pramila Vallabhaneni
Platform	Salesforce Developer Edition
Project Category	Automotive
Project Type	Individual
Org	Coral Cloud Resort (WhatNext Vision Motors app)
Report Date	September 20, 2026

Scope of this report: only Salesforce components supported by screenshots from the author's own Developer Edition org are described as implemented. Any remaining requirement not shown in evidence is identified as a requirement only, not as completed work.

Contents

Section	Title
1	Project Overview
2	Objectives and Scope
3	Data Model: Objects, Fields and Relationships
4	Lightning App and Custom Tabs
5	Process Automation
5.1	Auto Assign Dealer Flow
5.2	Test Drive Reminder Flow
5.3	Vehicle Stock Validation
6	Apex and Trigger
7	Batch Apex and Scheduled Apex
8	Testing and Evidence
9	Final System Architecture
10	Conclusion

1. Project Overview

1.1 Company and Project Context

WhatNext Vision Motors is an automotive company focused on transforming mobility through innovative technology and customer-focused solutions. This project implements a Salesforce CRM solution for the company as an individual project, built in a Salesforce Developer Edition org (Coral Cloud Resort tenant) under a Lightning application named “WhatNext Vision Motors”.

1.2 Problem Addressed

The company needs to manage vehicles, dealers, customers, orders and test drives in one system. The project addresses three business needs:

- Customer orders should be routed to a dealer automatically rather than assigned manually.
- Customers should not be able to place orders for vehicles that are out of stock.
- Customers who book a test drive should receive an automated email reminder before the scheduled date.

1.3 Salesforce Solution

The solution uses custom objects and relationships to model the automotive business, a Lightning application with custom tabs for navigation, Record-Triggered Flows for process automation, and an Apex trigger with a trigger handler, a Batch Apex class and a Schedulable Apex class for stock and order processing. The status of each component in this report is set out in Section 2.2.

1.4 Business Objectives

- Centralise vehicle, stock, dealer, customer, order and test-drive data in Salesforce.
- Reduce manual effort in assigning orders to dealers.
- Prevent orders for vehicles that are out of stock, both at the point of entry and through periodic processing.
- Improve customer communication through automated test-drive reminders.

2. Objectives and Scope

2.1 Project Requirements

The requirements below were used to guide the build. They are grouped by area.

Area	Requirement
Salesforce CRM implementation	Store and manage vehicle details, stock availability and dealer information; track customer orders and test drives.
Process automation	Prevent order placement if the vehicle is out of stock; auto-assign orders to a dealer; send automated email reminders for scheduled test drives.
Apex and trigger	Implement an Apex trigger to enforce stock validation and update stock on order placement, using a trigger handler for modularity.
Batch and scheduled jobs	Batch Apex job to process pending orders against vehicle stock; a Schedulable class to run the batch job on a schedule.

2.2 Implementation Status and Evidence

A requirement is described as implemented only where a screenshot supports it. Evidence levels used are: Evidenced (directly shown), Partly evidenced (some aspects shown) and Not evidenced (no supporting material, so it is treated as a requirement only).

Component	Status	Basis
Salesforce Developer Edition org	Evidenced	Developer Edition banner visible in all figures.
Four custom objects (Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order)	Evidenced	Records or edit forms shown for all four (Figs. 1–4, 6).
Vehicle Test Drive object	Partly evidenced	Referenced by the Test Drive Reminder flow and by the reminder email's record ID (Figs. 7–8); no record detail page captured.
Custom tabs and Lightning app	Evidenced	App navigation bar with all object tabs visible (Fig. 5).
Auto Assign Dealer flow	Partly evidenced	Flow built and the Save dialog captured (Fig. 5′/see §5.1); the resulting order shows an Assigned Dealer (Fig. 6). No activation confirmation screenshot was captured.
Test Drive Reminder flow	Evidenced	Flow activated (Fig. 7). A reminder email tied to the flow's scheduled path was received (Fig. 8).
Out-of-stock order validation	Evidenced	Validation error displayed on saving a third order for the same vehicle (Fig. 9).
Apex trigger and trigger handler	Evidenced	VehicleOrderTrigger and VehicleOrderTriggerHandler classes open in Developer Console (Figs. 10–11).
Batch Apex class	Partly evidenced	VehicleOrderBatch class shown querying Pending orders and vehicle stock (Fig. 12); the full execute/finish logic is not visible.
Schedulable Apex class	Evidenced	VehicleOrderBatchScheduler class shown scheduling VehicleOrderBatch with a batch size of 50 (Fig. 13).
Scheduled Jobs entry / batch execution log	Not evidenced	No Scheduled Jobs list or Apex Job history screenshot provided.

3. Data Model: Objects, Fields and Relationships

The solution is built on custom objects for vehicles, dealers, customers and orders, plus a Vehicle Test Drive object referenced by the process-automation flows. Field names below are the labels displayed in the org; underlying API names visible in the Apex code (Section 6) are noted where relevant (e.g. Vehicle_Order__c, Vehicle__c, Status__c).

Object	Purpose	Fields shown in evidence	Relationships shown in evidence
Vehicle Customer	Stores customer contact and preference details.	Vehicle Customer Name, Email, Phone, Address, Preferred Vehicle Type (Fig. 1).	Referenced by Vehicle Order and by the Test Drive Reminder flow.
Vehicle Dealer	Stores dealer identity and location, used for order assignment.	Vehicle Dealer Name, Dealer Location, Dealer Code, Phone, Email (Figs. 2–3).	Referenced by Vehicle (Vehicle Dealer field) and by Vehicle Order (Assigned Dealer field).
Vehicle	Stores vehicle details and stock availability.	Vehicle Name, Vehicle Model, Stock Quantity, Price, Status (Fig. 4).	Vehicle Dealer relationship field (Fig. 4); referenced by Vehicle Order.
Vehicle Order	Records a customer's order for a vehicle.	Vehicle Order Number, Order Date, Status (Fig. 6).	Vehicle Customer, Vehicle and Assigned Dealer fields (Fig. 6).
Vehicle Test Drive	Records scheduled test drives used by the reminder flow.	Not shown as a record page in the supplied screenshots.	Referenced by the Test Drive Reminder flow's Get Customer Information step (Fig. 7) and by the reminder email's record ID (Fig. 8).

3.1 Vehicle Customer

The Vehicle Customer record “John” stores Name, Email, Phone, Address and Preferred Vehicle Type. The Address field (Hyderabad) is the location field the ordering and dealer-assignment logic can draw on.

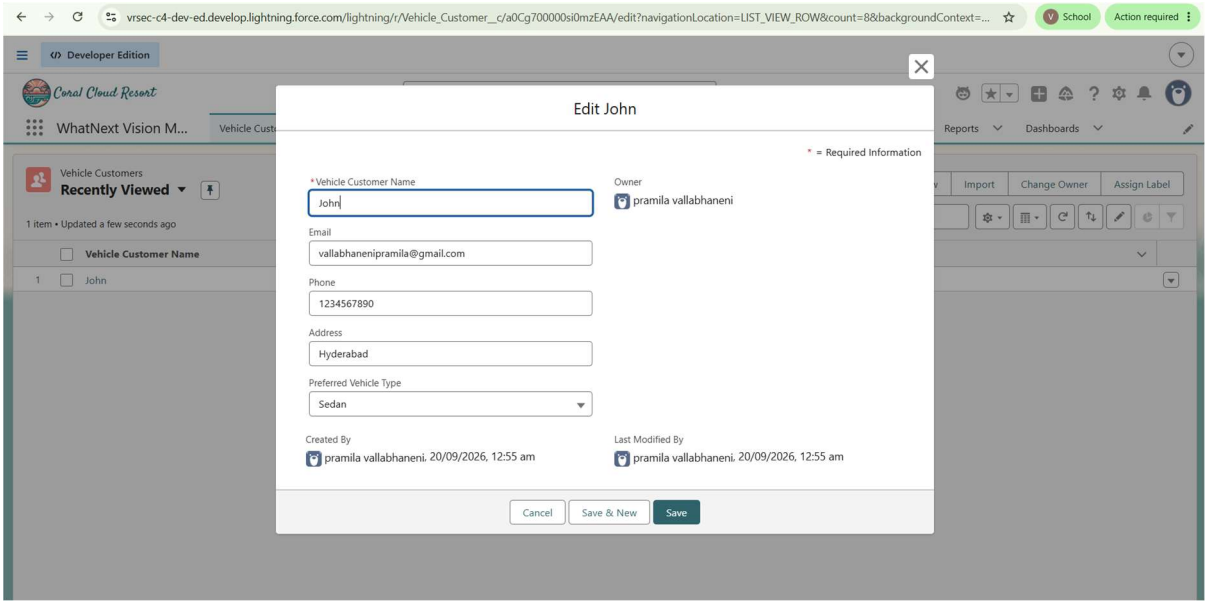


Figure 1: Vehicle Customer record “John”, showing Name, Email, Phone, Address (Hyderabad) and Preferred Vehicle Type (Sedan).

3.2 Vehicle Dealer

Two Vehicle Dealer records were created: “Em” (Dealer Code DC-0001, Hyderabad) and “TM” (Dealer Code DC-0002, Vizag). Figure 2 shows the “Em” record; the “TM” record was created and edited with the equivalent fields.

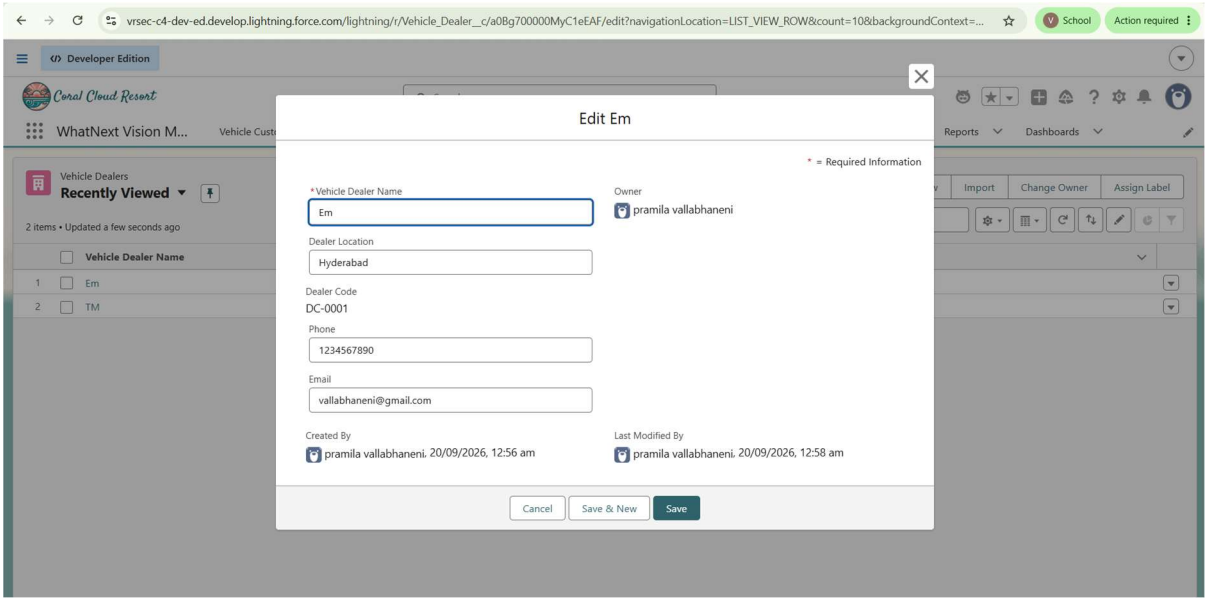


Figure 2: Vehicle Dealer record “Em”, Dealer Code DC-0001, Dealer Location Hyderabad.

3.3 Vehicle

The Vehicle record “Honda” stores Vehicle Model (EV), Stock Quantity (100 at creation), Price (₹80,000), a relationship to Vehicle Dealer (TM) and Status (Available). Stock Quantity and Status hold the stock-availability information the ordering requirements depend on.

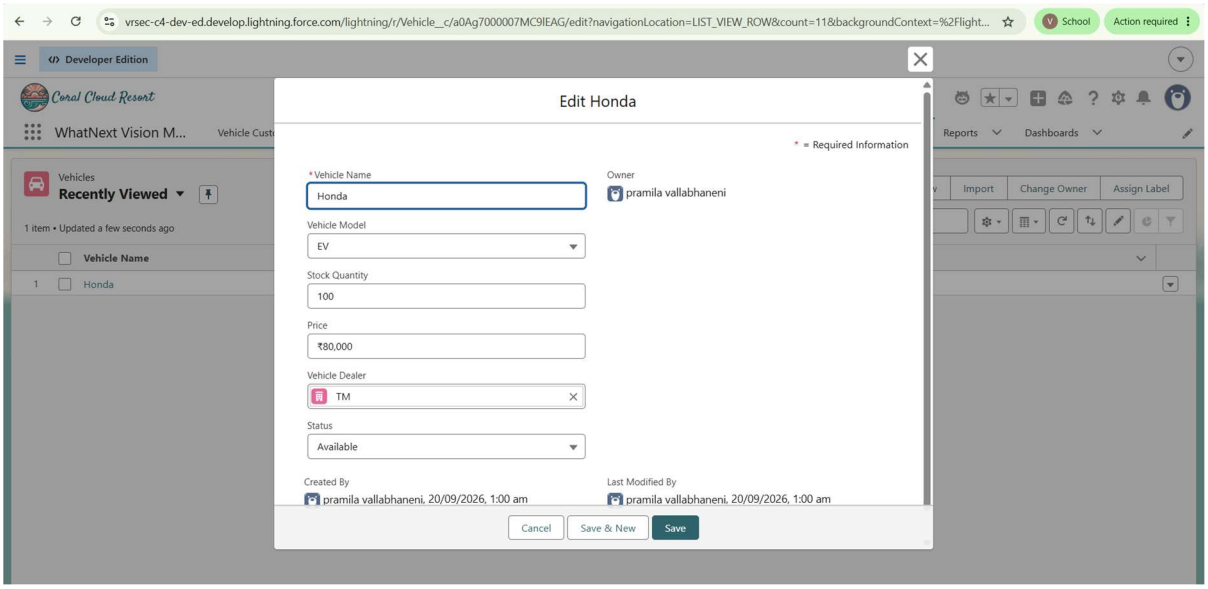


Figure 3: Vehicle record “Honda” showing Vehicle Model, Stock Quantity, Price, Vehicle Dealer and Status fields.

3.4 Vehicle Order

The Vehicle Order object links a Vehicle Customer and a Vehicle, and records Order Date, Status and Assigned Dealer. Order O-0001 (Fig. 6, Section 5.1) demonstrates all of these fields populated, including an automatically assigned dealer.

4. Lightning App and Custom Tabs

A Lightning application named “WhatNext Vision Motors” provides the working area for the CRM. Its navigation bar contains tabs for Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles, Reports and Dashboards, giving users access to every project object from one application.

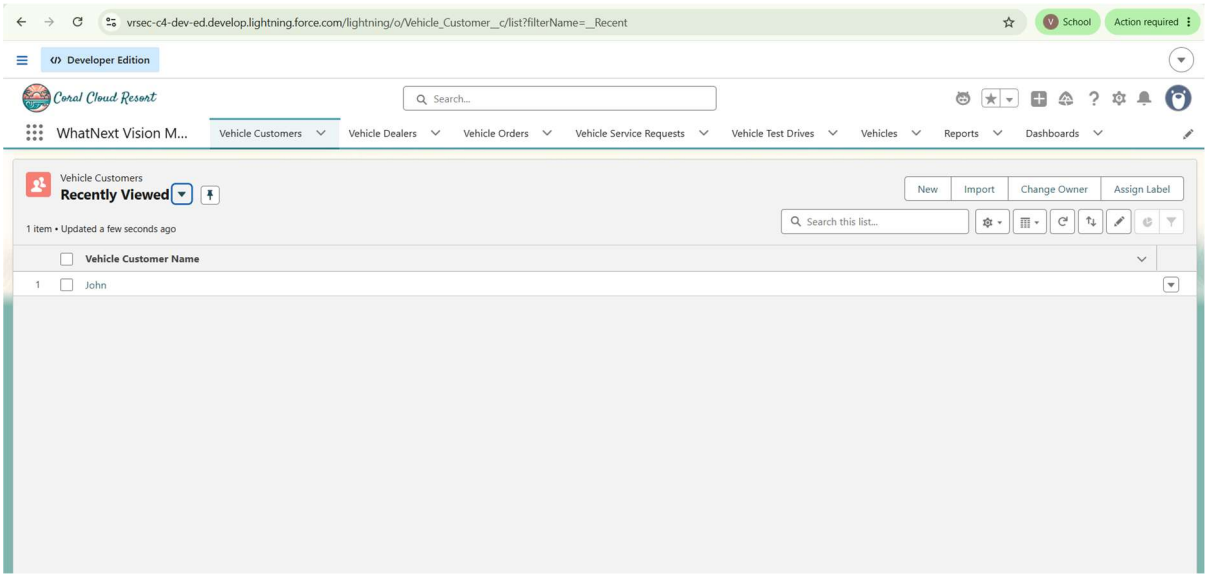


Figure 4: Vehicle Customers list view within the WhatNext Vision Motors app, showing the full tab navigation bar.

Evidence limits: the Setup > Tabs configuration screen (showing each Custom Object Tab's style and description) was not included in the evidence; the app and its navigation are evidenced by the running application instead.

5. Process Automation

Two Record-Triggered Flows were built in Flow Builder, and an out-of-stock validation was observed on the Vehicle Order form. Only the configuration visible in the screenshots is documented.

5.1 Auto Assign Dealer Flow

This flow supports the requirement to assign orders to a dealer automatically rather than manually. Figure 5 shows the flow's structure and the Save dialog in Flow Builder.

Setting	Observed configuration
Flow name	Auto Assign Dealer (API name Auto_Assign_Dealer)
Flow type	Record-Triggered Flow
Start path	Run Immediately
Element 1	Get Customer Information (Get Records)
Element 2	Get Nearest Dealer (Get Records)
Element 3	Assign Dealer to Order (Update Records)
Save captured	02:18, 20/09/2026 — Save the Flow dialog with Flow Label and API Name entered.

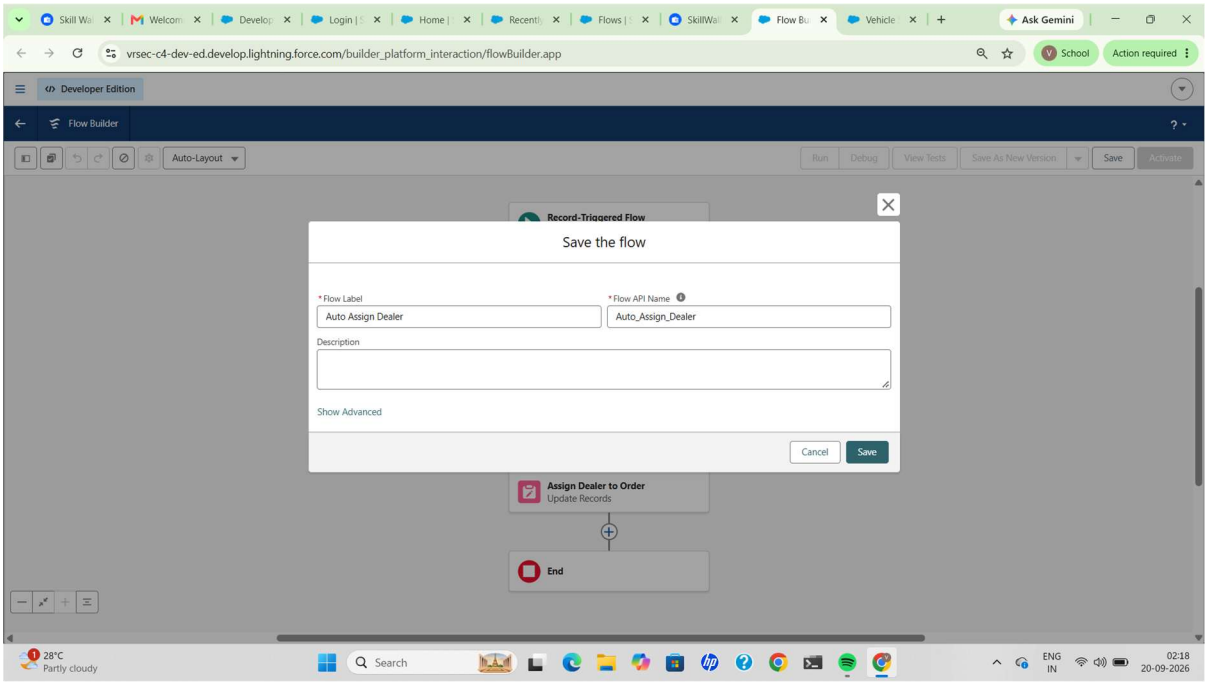


Figure 5: Auto Assign Dealer flow — structure (Get Customer Information → Get Nearest Dealer → Assign Dealer to Order) and the Save the Flow dialog.

The flow runs immediately after the triggering record change. It retrieves customer information, retrieves a dealer record, and updates a record to assign the dealer to the order. Order O-0001 (Fig. 6) shows an Assigned Dealer value of “Em”, which is consistent with this flow having run.

Evidence limits: the trigger object, entry conditions, the filter used to select the nearest dealer, and a screenshot of the flow's Active status toggle are not captured. The link between the flow and the populated Assigned Dealer field is therefore made by consistency of outcome rather than a visible execution log.

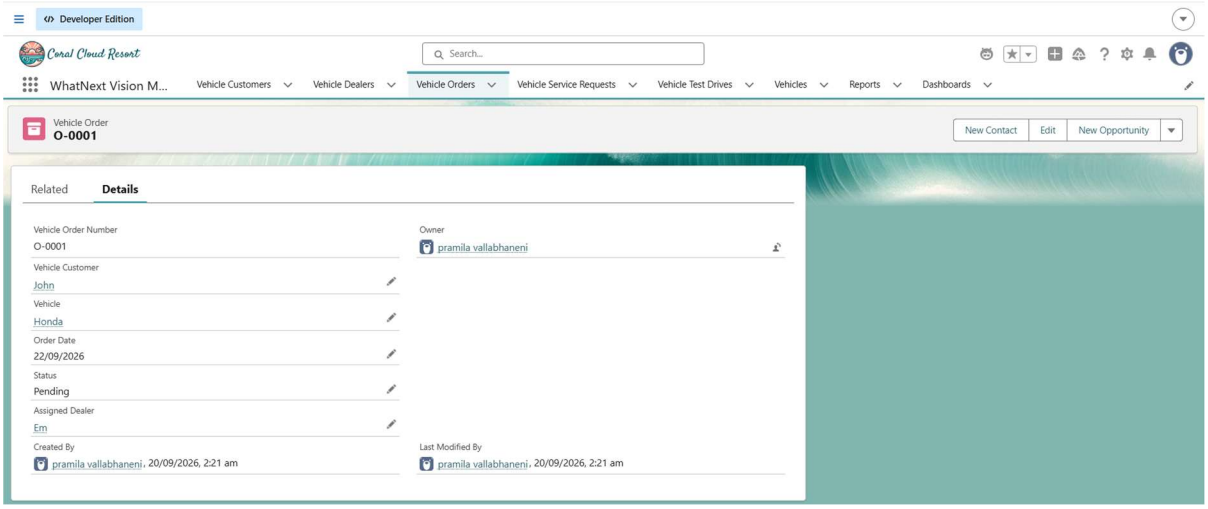


Figure 6: Vehicle Order O-0001, showing Vehicle Customer John, Vehicle Honda, Status Pending and Assigned Dealer Em.

5.2 Test Drive Reminder Flow

This flow supports the requirement to send automated email reminders for scheduled test drives. It uses a scheduled path so the reminder is not sent at the moment the record is saved.

Setting	Observed configuration
Flow name	Test Drive Reminder – V1
Flow type	Record-Triggered Flow
Run Immediately path	Ends without performing any actions
Scheduled path	Reminder Before Test Drive
Scheduled path elements	Get Customer Information (Get Records), then Send Test Drive Reminder (Action)
Status	Active. Last saved 20/09/2026, 02:34 am.

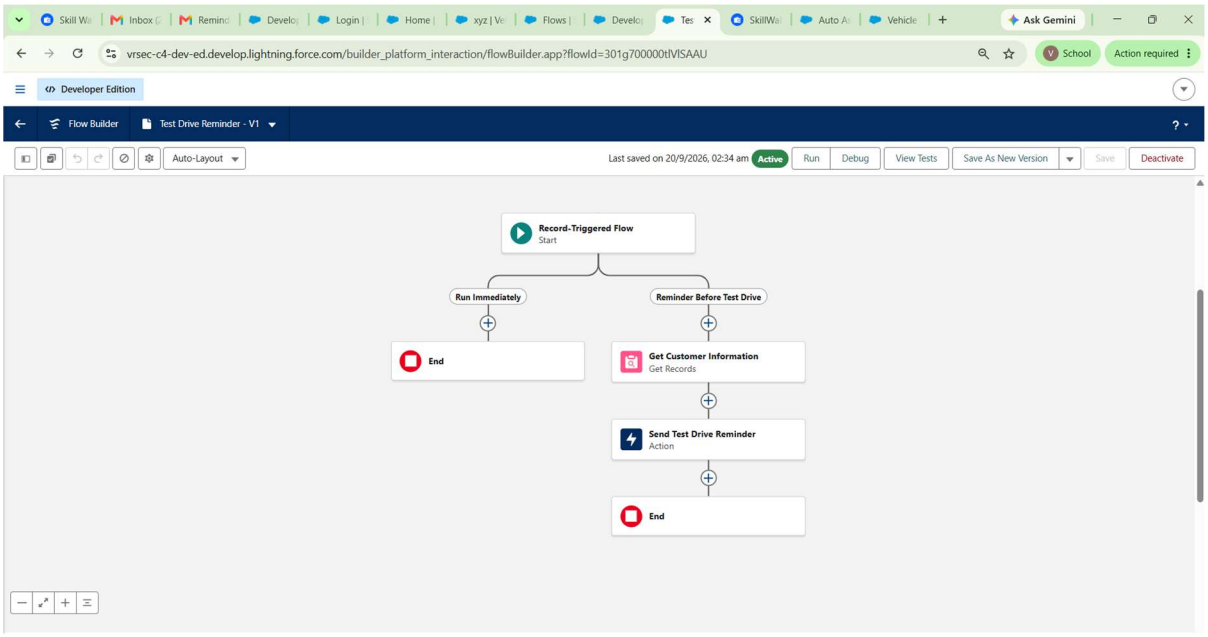


Figure 7: Test Drive Reminder – V1, Active, showing the Run Immediately and Reminder Before Test Drive paths.

On the scheduled path the flow retrieves the customer information and then runs the Send Test Drive Reminder action. A reminder email tied to a test drive record was received shortly after activation, confirming the flow's output end to end.

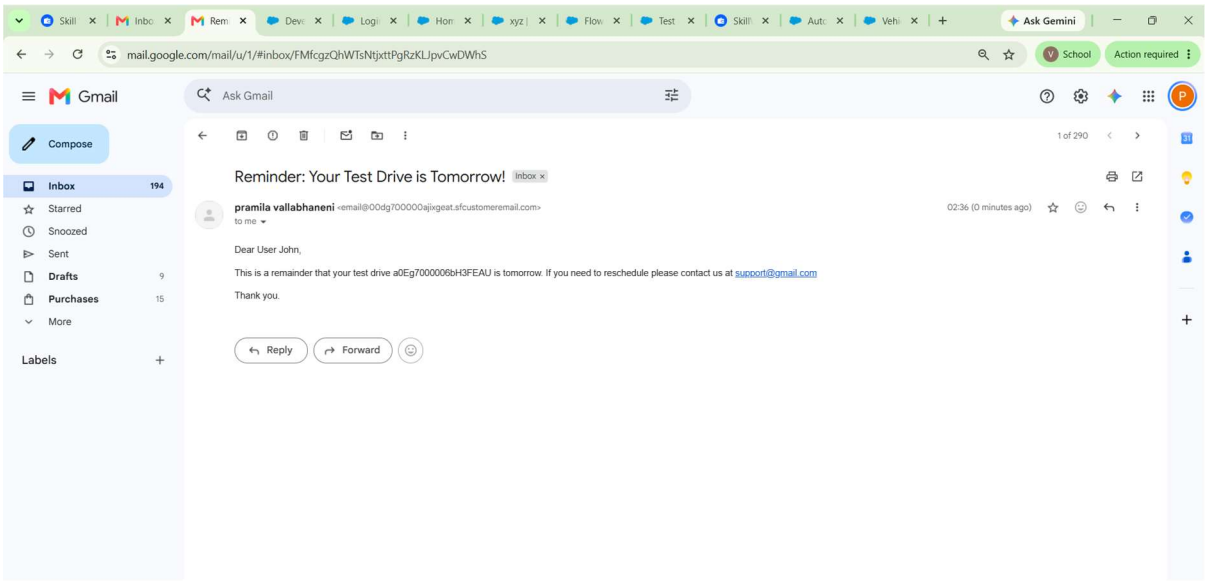


Figure 8: Reminder email received in Gmail, "Reminder: Your Test Drive is Tomorrow!", referencing test drive record a0Eg7000006bH3FEAU and addressed to "User John."

Evidence limits: no Vehicle Test Drive record detail page was captured, so the Test Drive Date and Status of the underlying record are known only through the flow's Get Customer Information step and the record ID referenced in the email. The flow's run history (Debug or Paused/Waiting Interviews) was not captured, so the email is linked to the flow by timing and by the record ID in its body rather than by a visible flow-run log.

5.3 Vehicle Stock Validation

The requirement is to prevent order placement when a vehicle is out of stock. Two Vehicle Orders (O-0001 and O-0002) were successfully created for the vehicle Honda. A third attempt to create an order for the same vehicle, with Vehicle Customer John, Order Date 22/09/2026 and Status Pending, was blocked.

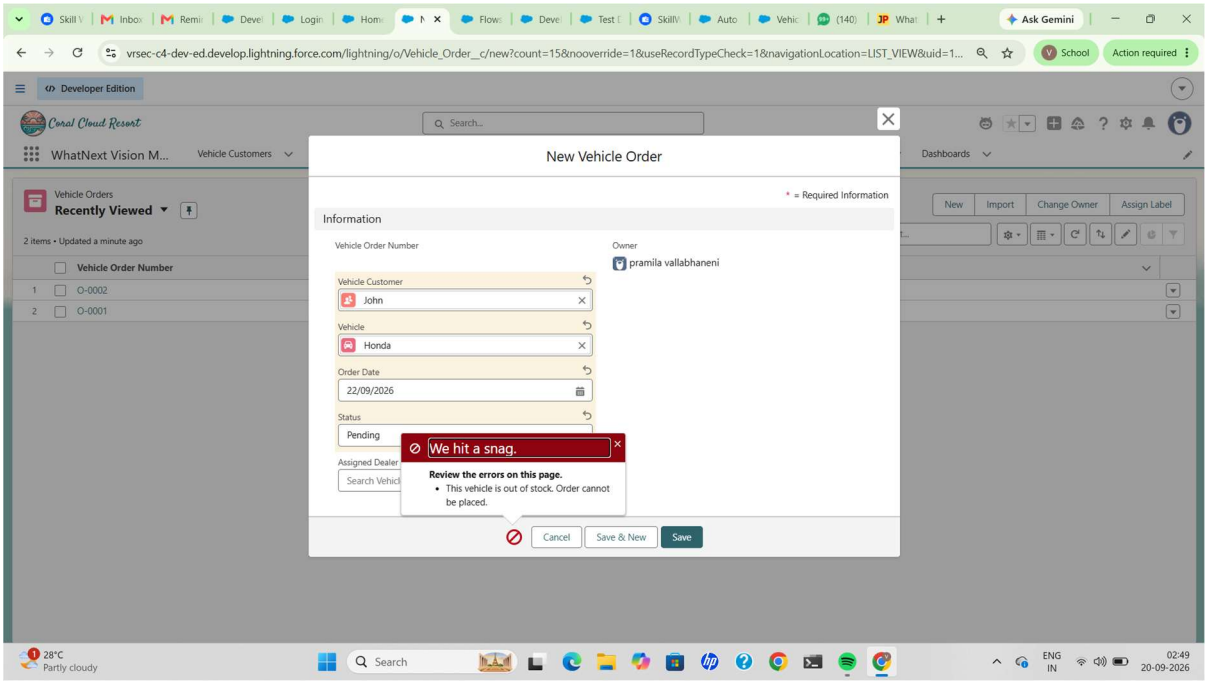


Figure 9: "We hit a snag" validation error — "This vehicle is out of stock. Order cannot be placed." — displayed when saving a third Vehicle Order for Honda.

Salesforce responded with the message "We hit a snag" and the error "This vehicle is out of stock. Order cannot be placed.", and the New Vehicle Order form remained open. Section 6 shows the Apex trigger logic (preventOrderIfOutOfStock) that this behaviour is consistent with.

Evidence limits: the Stock Quantity of the Honda vehicle record at the moment of this attempt was not captured in a screenshot, so the exact stock level that triggered the block is not documented here.

6. Apex and Trigger

An Apex trigger and a separate trigger handler class were found open in the Developer Console, implementing the stock-validation and stock-update logic referenced in Section 5.3.

6.1 VehicleOrderTrigger

The trigger is defined on Vehicle_Order__c for all four trigger events and delegates all logic to the handler class, following the one-trigger-per-object pattern.

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert,
after update) {
    VehicleOrderTriggerHandler.handleTrigger(Trigger.new, Trigger.oldMap, Trigger.isBefore,
        Trigger.isAfter, Trigger.isInsert, Trigger.isUpdate);
}
```

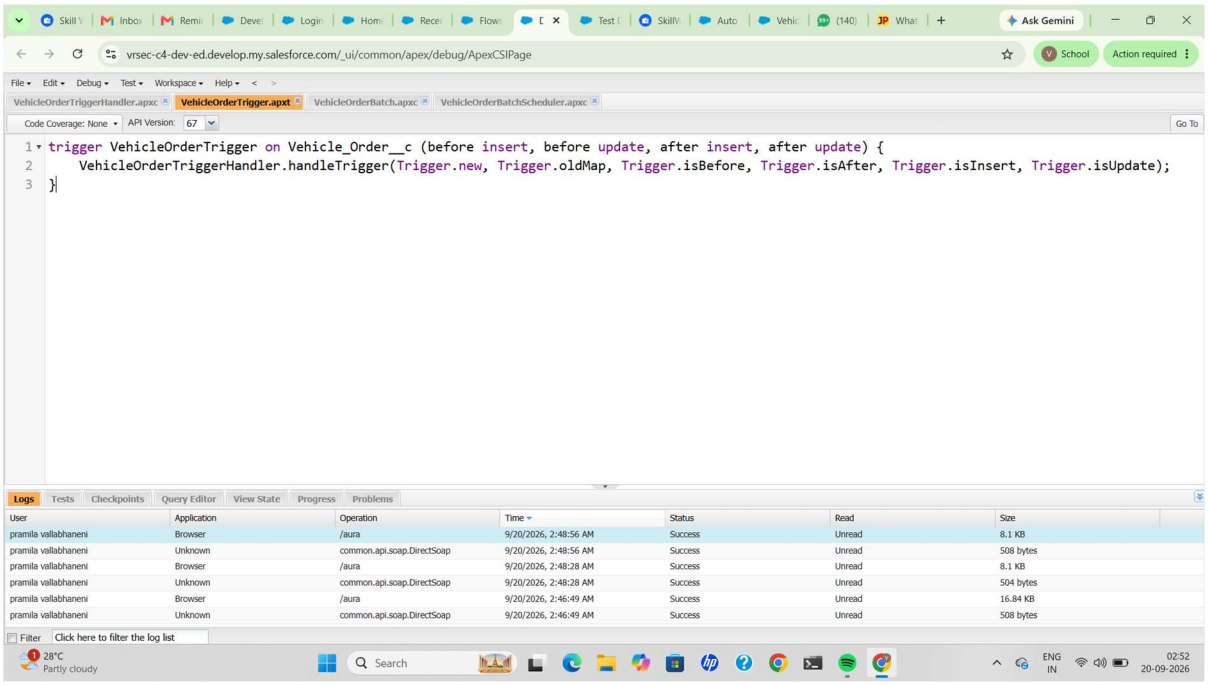


Figure 10: VehicleOrderTrigger.apxt, open in Developer Console.

6.2 VehicleOrderTriggerHandler

The handler routes before-save events to a stock-validation method and after-save events to a stock-update method, keeping the two concerns separate.

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
        Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, ...) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }
        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }

    // Prevent placing an order if stock is zero
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
            // ... (remainder not captured in the screenshot)
```

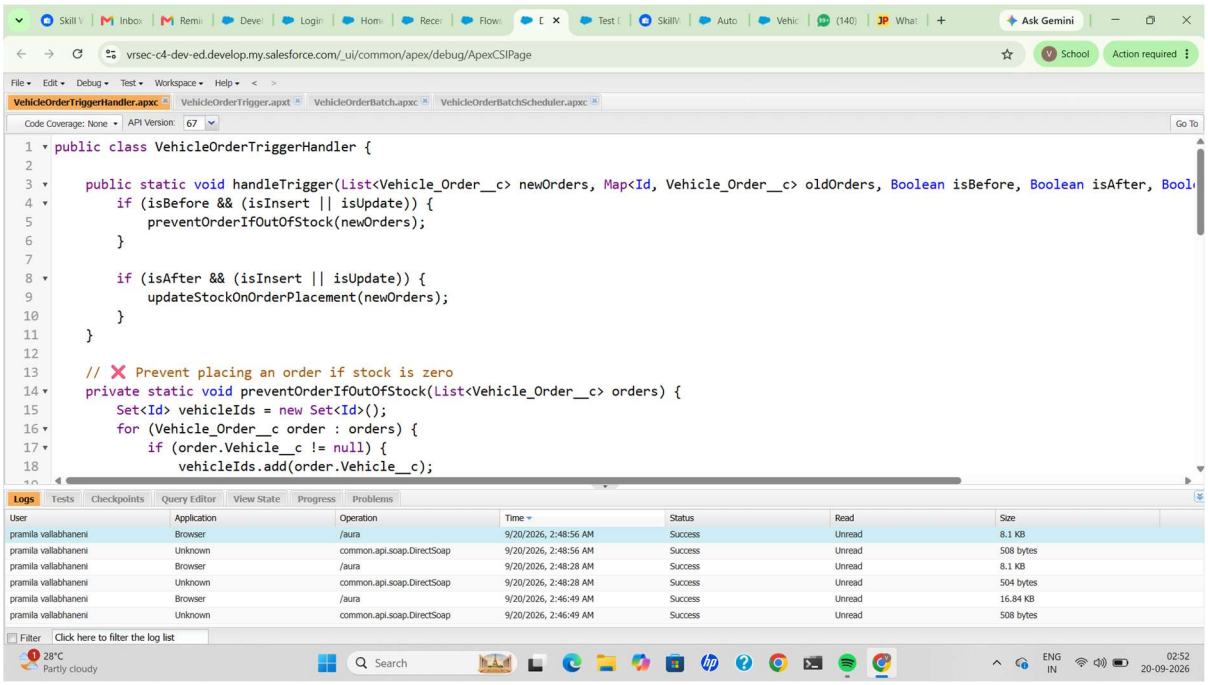


Figure 11: VehicleOrderTriggerHandler.apxc, open in Developer Console, showing handleTrigger and the start of preventOrderIfOutOfStock.

Evidence limits: the body of preventOrderIfOutOfStock beyond line 18 and the full updateStockOnOrderPlacement method were not captured, so the exact stock-comparison logic and the amount by which stock is decremented are not documented.

7. Batch Apex and Scheduled Apex

A Batch Apex class and a Schedulable class were also found open in the Developer Console, addressing the requirement to process pending orders against vehicle stock on a schedule.

7.1 VehicleOrderBatch

The batch job queries all Vehicle Orders with Status ‘Pending’, collects their related Vehicle Ids, and begins building a map of vehicle stock for those Ids.

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c FROM Vehicle_Order__c WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (!vehicleIds.isEmpty()) {
            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>(
```

```
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]
        // ... (remainder not captured in the screenshot)
```

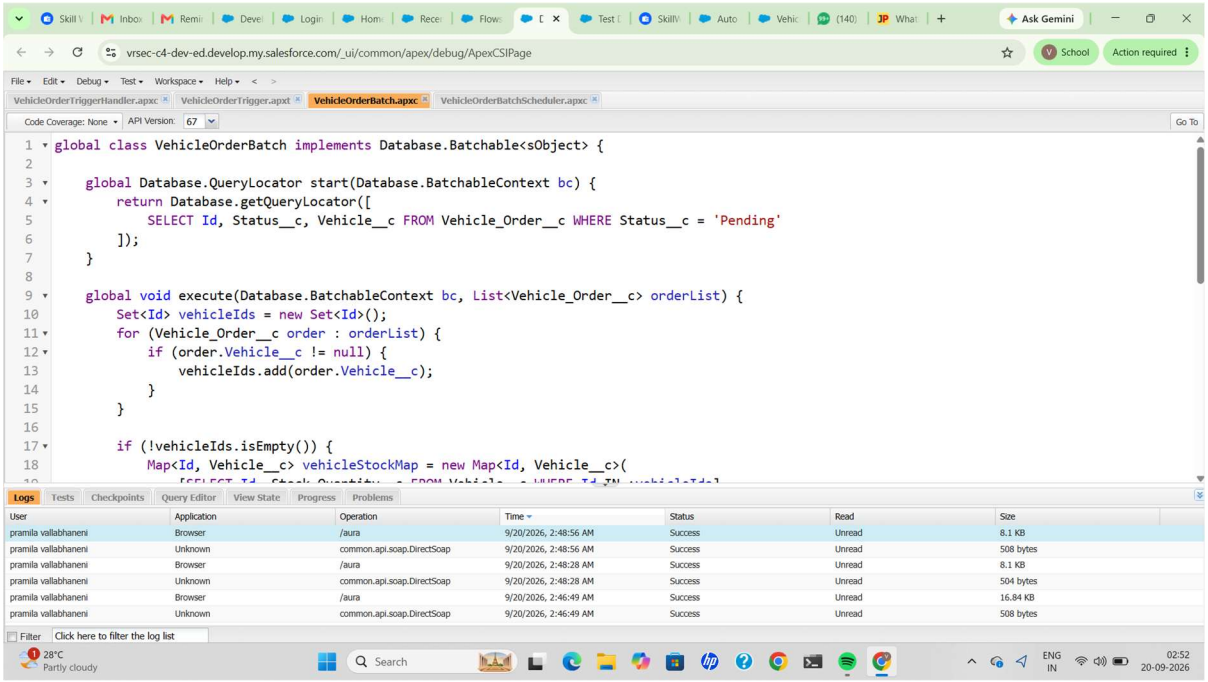


Figure 12: VehicleOrderBatch.apxc, open in Developer Console, showing start() and the beginning of execute().

Evidence limits: the remainder of execute() and the finish() method are not visible in the screenshot, so how Status is updated (Pending vs Confirmed) and how stock is written back are not documented from this evidence alone; they are inferred only from the class's purpose as named.

7.2 VehicleOrderBatchScheduler

A Schedulable class runs VehicleOrderBatch with a batch size of 50.

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 = batch size
    }
}
```

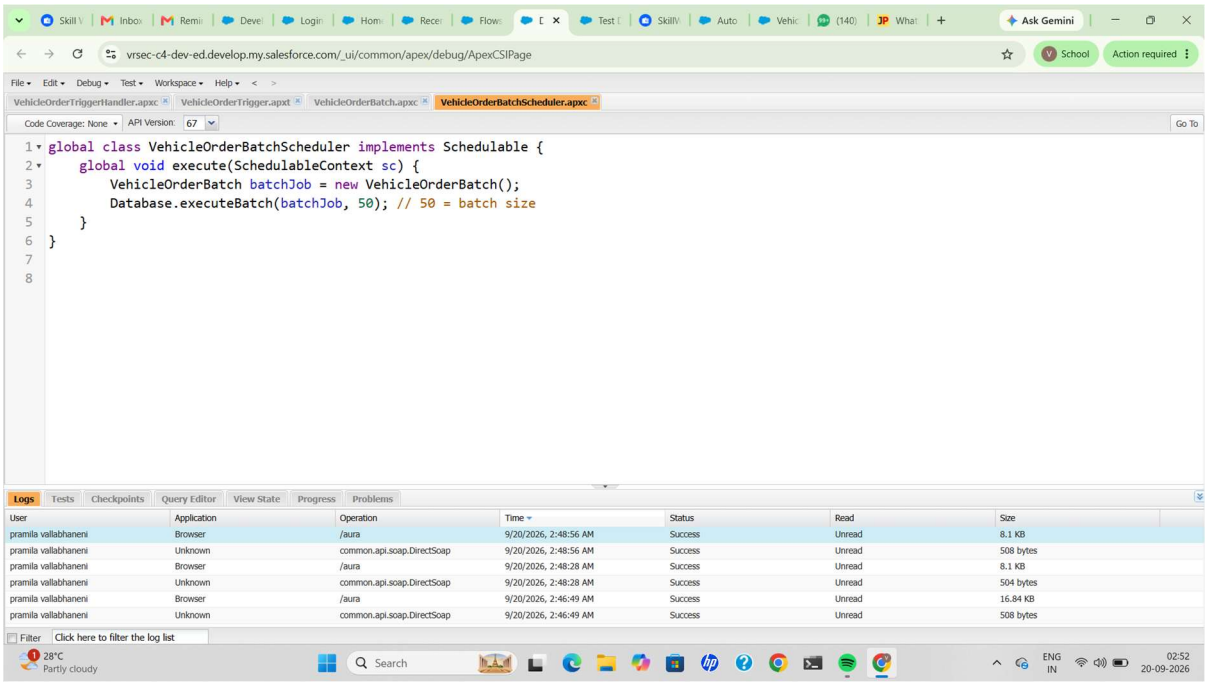


Figure 13: VehicleOrderBatchScheduler.apxc, open in Developer Console.

Evidence limits: no Scheduled Jobs list entry or Apex Job history screenshot was provided, so it is not documented whether VehicleOrderBatchScheduler has actually been scheduled (e.g. via System.schedule) or executed in the org. The Developer Console's "Code Coverage: None" indicator on each tab also shows no test run had been executed against these classes at the time of the screenshots.

8. Testing and Evidence

8.1 Tests Supported by Evidence

Test case	Input / condition	Expected result	Actual result	Result
TC-01 Object records	Create Vehicle Customer, Vehicle Dealer (x2) and Vehicle records.	Records save with the expected fields.	Records "John", "Em", "TM" and "Honda" created and shown with populated fields.	Verified
TC-02 App navigation	Open the WhatNext Vision Motors app.	All object tabs are visible.	Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles, Reports and Dashboards all shown.	Verified
TC-03 Dealer auto-assignment	Save an order for customer John and vehicle Honda.	Order is created with a dealer automatically assigned.	Order O-0001 shows Assigned Dealer = Em.	Consistent with expected outcome
TC-04 Test Drive	Activate the flow in Flow Builder.	Flow status is Active.	Status Active; last saved 02:34 am.	Verified

Test case	Input / condition	Expected result	Actual result	Result
Reminder activation				
TC-05 Test drive reminder delivery	Test drive record referenced by ID a0Eg7000006bH3FEAU.	A reminder email is received before the test drive date.	Email “Reminder: Your Test Drive is Tomorrow!” received in Gmail, addressed to “User John” and referencing the record ID.	Verified
TC-06 Successful orders (in stock)	Place two orders (O-0001, O-0002) for Honda.	Orders save successfully while stock is available.	Both orders created and listed.	Verified
TC-07 Out-of-stock order blocked	Attempt a third order for Honda.	Order is blocked with an out-of-stock message.	Error “This vehicle is out of stock. Order cannot be placed.” displayed; form stayed open.	Verified
TC-08 Apex trigger / handler present	Open Developer Console.	Trigger and handler classes exist and compile.	VehicleOrderTrigger and VehicleOrderTriggerHandler open with no compile errors shown.	Verified (structure only)
TC-09 Batch and Scheduler classes present	Open Developer Console.	Batch and Schedulable classes exist and compile.	VehicleOrderBatch and VehicleOrderBatchScheduler open with no compile errors shown.	Verified (structure only)

Note: TC-03 is verified by consistency of outcome rather than a visible flow-run log (see §5.1). TC-08 and TC-09 verify that the classes exist and are syntactically valid in the Developer Console; they do not verify execution, scheduling, or test coverage.

8.2 Requirements Without Test Evidence

- Vehicle stock value at the moment the out-of-stock error was triggered.
- Scheduled Jobs entry showing VehicleOrderBatchScheduler actually scheduled.
- Apex Job history or debug log showing VehicleOrderBatch executing against Pending orders.
- Unit test classes and code coverage percentage for the trigger, batch and scheduler classes.
- A Vehicle Test Drive record detail page (Test Drive Date, Status) shown directly.

9. Final System Architecture

The diagram below summarises the implemented system as two processes: order handling and test-drive reminders.

Order Process	Test Drive Process
Vehicle Customer Customer record with Address (Fig. 1)	Vehicle Customer Customer record with Email
Vehicle Order New order form: Customer, Vehicle, Order Date, Status (Fig. 6, 9)	Vehicle Test Drive Referenced by record ID in reminder email (Fig. 8)

Order Process	Test Drive Process
Auto Assign Dealer Flow Get Customer Information → Get Nearest Dealer → Assign Dealer to Order (Fig. 5)	Test Drive Reminder Flow Scheduled path: Reminder Before Test Drive (activated, Fig. 7)
Stock Validation (Apex Trigger) preventOrderIfOutOfStock blocks the save when stock is zero (Figs. 9–11)	Scheduled Path Actions Get Customer Information, then Send Test Drive Reminder action (Fig. 7)
Stock Update (Apex Trigger) updateStockOnOrderPlacement runs after insert/update (Fig. 11)	Email Notification Delivered Reminder email received (Fig. 8)
Batch Apex / Scheduler VehicleOrderBatch queries Pending orders; VehicleOrderBatchScheduler runs it on a schedule (Figs. 12–13)	

9.1 Summary of the Implemented System

- Vehicle, Vehicle Dealer, Vehicle Customer and Vehicle Order objects with relationship fields store the automotive CRM data; a Vehicle Test Drive object is used by the reminder flow.
- The WhatNext Vision Motors Lightning app exposes the object tabs, Reports and Dashboards from one navigation bar.
- The Auto Assign Dealer flow is built from Get Customer Information, Get Nearest Dealer and Assign Dealer to Order elements; an order created after its build shows a dealer assigned.
- The Test Drive Reminder flow is activated and contains a scheduled path, Reminder Before Test Drive, with a Get Records element and a Send Test Drive Reminder action; its output was confirmed by a received reminder email.
- An Apex trigger (VehicleOrderTrigger) and trigger handler (VehicleOrderTriggerHandler) implement before-save stock validation and after-save stock updates; the validation behaviour was observed directly (Fig. 9).
- A Batch Apex class (VehicleOrderBatch) and a Schedulable class (VehicleOrderBatchScheduler, batch size 50) exist to process Pending orders against vehicle stock on a schedule.

10. Conclusion

This project delivers the core CRM foundation for WhatNext Vision Motors in a Salesforce Developer Edition org. Custom objects model vehicles, dealers, customers and orders, with relationship fields linking orders to customers, vehicles and dealers. Custom tabs and the WhatNext Vision Motors Lightning app give users access to these records.

Automation is provided by two Record-Triggered Flows: Auto Assign Dealer, which retrieves customer and dealer records and updates the order, and Test Drive Reminder, which uses a scheduled path to send a reminder for upcoming test drives — confirmed end to end by a received reminder email. Vehicle stock handling is additionally backed by Apex: a trigger and handler pair block orders when stock is zero and update stock after an order is placed, and a Batch Apex class paired with a Schedulable class is in place to process pending orders against stock on a recurring schedule.

The main gaps in the evidence are a Scheduled Jobs entry confirming the scheduler has actually been scheduled, an execution or debug log confirming a batch run, unit test coverage for the Apex classes, and a direct view of a Vehicle Test Drive record. This report describes only what the author's Salesforce org evidence supports.